

SmartStock AI

Graduation Project — Technical Specification Report

Team Name: Horizonte Infinito

Target Industry: Logistics, E-commerce, and Retail Supply Chain

Team Members: Ayman Mohamed · Omar Wael · Mostafa Abdel Aziz · Mostafa Abdel Qawy · Mawada Alexander

June 2025

Section A: Use Case & Product Definition

A.1 Problem Statement

E-commerce and logistics enterprises sustain substantial financial losses attributable to two antithetical inventory failure modes: overstocking, which immobilises working capital in idle goods, and stockouts, which generate lost sales and erode customer satisfaction. Incumbent inventory management systems are inherently reactive, relying on periodic manual audits that fail to incorporate macroeconomic signals, seasonal demand curves, or supplier lead-time variability.

A.2 Solution & Value Proposition

SmartStock AI is an integrated, web-based warehouse management platform that couples real-time operational data capture with AI-driven demand forecasting. Its core value proposition — Proactive Demand Planning — shifts decision-making from reactive correction to anticipatory action, notifying warehouse managers of impending shortfalls before they materialise, thereby minimising waste and maximising profitability.

A.3 Target Users

- Primary: Warehouse managers and owners of medium-to-large e-commerce businesses.
- Secondary: Procurement teams and supply chain analysts requiring data-driven reorder insights.
-

A.4 Success Metrics (KPIs)

To ensure measurable business impact, SmartStock AI defines the following KPIs:

1- Stockout Reduction Rate

- Target: Reduce stockout incidents by **30% within 3 months** of deployment.
- Formula:

1- Purchase Order Automation Rate

- Target: At least **60% of POs auto-drafted and approved without manual edits**.
- Measures efficiency of Purchasing Agent.

1- Inventory Carrying Cost Reduction

- Target: Reduce excess inventory holding cost by **15% in 6 months**.

A.5 Quantified Market Pain

- SMEs typically spend **20–30% of revenue on inventory**.
- Overstocking increases holding cost by **15–25% annually**.
- Stockouts reduce revenue by **4–8% per year** (Retail Industry Benchmarks).

This validates the economic urgency of proactive demand planning.

A.6 4-Week MVP Delivery Roadmap

Week 1 — Core Inventory System

- Inventory CRUD (Products, Suppliers, Stock Levels)
- Dashboard with:
 - Current stock
 - Low stock alerts
- PostgreSQL schema finalized
- Basic authentication

Owner: Backend + Frontend Team

Week 2 — Demand Forecasting Engine

- Prophet forecasting model (using prophet Django ORM Pandas libraries)
- Historical sales ingestion
- Reorder threshold logic
- Display forecast chart in dashboard

Owner: ML Engineer

And the architecture of this process will be like this

PostgreSQL



Django ORM



Pandas



Prophet



Forecast Logic



REST API



React + Recharts

Week 3 — LLM Natural Language Analytics

- GPT-4o integration
- Prompt template implementation
- Function calling for structured JSON output
- 5 few-shot NL-to-query examples
- Secure API key handling via .env

Owner: AI Engineer

Frontend (React Chat UI)



Django REST Endpoint



LangChain Prompt Template



GPT-4o (Function Calling)



Structured JSON



Database Query



Response to User

Week 4 — Purchasing Agent (Human-in-the-Loop)

- Agent ReAct loop
- PO auto-drafting
- Email integration
- Approval workflow
- Error handling (retry + timeout)

Owner: AI + Backend

Section B: AI Core — LLM & Prompt Engineering

B.1 System Prompt Template

text

You are SmartStock AI, a warehouse inventory analytics assistant.

Your role:

- Translate user natural language queries into structured database queries.
- Only operate within inventory, suppliers, sales, and purchase orders.
- Never generate free-form SQL.
- Always respond using the provided JSON schema.

If the request is outside inventory scope, respond:

```
{"error": "Out of scope request"}
```

B.2 Structured Output via Function Calling

JSON Schema

JSON

```
{  
  "type": "object",  
  "properties": {  
    "action": {  
      "type": "string",  
      "enum": ["get_inventory", "get_sales_report", "get_low_stock", "forecast_demand"]  
    },  
    "filters": {  
      "type": "object",  
      "properties": {  
        "product_name": {"type": "string"},  
        "date_from": {"type": "string"},  

```

```
"date_to": {"type": "string"},  
"stock_below": {"type": "number"}  
}  
  
},  
  
"required": ["action"]  
}
```

B.3 Example NL → Structured Output

Example 1

User:

"Show me low stock products"

LLM Output:

JSON

```
{  
  
  "action": "get_low_stock",  
  
  "filters": {  
  
    "stock_below": 10  
  
  }  
}
```

Example 2

User:

"Generate a sales report for March 2026"

JSON

```
{  
  
  "action": "get_sales_report",
```

```
"filters": {  
  "date_from": "2026-03-01",  
  "date_to": "2026-03-31"  
}  
}
```

Example 3

User:

"Forecast demand for Product X next month"

JSON

```
{  
  "action": "forecast_demand",  
  "filters": {  
    "product_name": "Product X"  
  }  
}
```

B.4 Few-Shot Prompting Strategy

Inside the system:

text

Example 1:

User: Show items with stock below 5

Output:

```
{"action": "get_low_stock", "filters": {"stock_below": 5}}
```

Example 2:

User: Sales report from Jan 1 to Jan 15

Output:

```
{"action": "get_sales_report", "filters": {"date_from": "YYYY-MM-DD", "date_to": "YYYY-MM-DD"}}
```


Section C: RAG Foundation

C.1 — Architecture Overview

The Retrieval-Augmented Generation (RAG) layer connects the LLM to internal business knowledge — including product catalogues, supplier records, shipping policies, and historical sales data — by extending the existing PostgreSQL database with the pgvector extension. This approach achieves an integrated, low-overhead vector search capability without introducing a separate external database service, keeping the infrastructure lean and operationally simple.

C.2 — Document Ingestion & Chunking Strategy

During the ingestion phase, all data sources are processed according to their structural type. Unstructured documents such as supplier PDFs and shipping policy files are partitioned into chunks of 512 tokens with a 50-token overlap using recursive text splitting, preserving contextual continuity across chunk boundaries. Structured database records — such as product catalogue rows and supplier entries — are chunked at the record level, ensuring each row is treated as a single atomic unit to prevent data corruption across fields.

Every text chunk is paired with an immutable metadata payload before being embedded and stored. This payload captures: the source document name, page number, record type, and ingestion timestamp. These metadata fields are later used to generate source citations in every AI response.

All chunks are subsequently vectorised into 1536-dimensional embeddings using OpenAI's text-embedding-3-small model (or text-embedding-ada-002 as a fallback) and stored in the pgvector index within PostgreSQL.

C.3 — Retrieval Pipeline

When a warehouse manager submits a natural language query, the backend executes a hybrid search that fuses two complementary retrieval strategies. Dense semantic vector matching captures the thematic intent of the query — for example, identifying that "items running low" relates to stock level records. Sparse PostgreSQL full-text keyword indexing runs in parallel to capture exact alphanumeric matches such as SKU codes or supplier names that semantic search alone may miss.

The combined candidate results are then passed through a Cohere cross-encoder reranking step (or a locally hosted Hugging Face bge-reranker model as an alternative) which scores each candidate against the query and isolates the top three highest-relevance context chunks. These three chunks are injected into the LLM context window for response generation.

C.4 — Source Citation & Hallucination Prevention

The system prompt governing the Natural Language Data Analyst strictly mandates source tracing on every response. The LLM is instructed:

"You are an assistant reading official warehouse documentation. You must answer the user's question using ONLY the provided context chunks. For every claim or statement you make, you must explicitly cite the source_document and page_number from the context metadata using square brackets — for example, [Source: filename.pdf, Page: X]. If the context does not contain the answer, explicitly state that you cannot find the information in the provided records. Do not attempt to invent an answer."

This instruction eliminates hallucination risk by grounding every response in verified enterprise data, and delivers fully auditable AI insights to the warehouse manager — every claim is traceable back to its original source document.

Section D: Agentic AI & Tool Use

SmartStock AI implements a multi-agent architecture orchestrated via LangChain, composed of three specialized autonomous agents that collaborate to execute the full demand-response pipeline.

Agent 1 — Forecasting Agent

The Forecasting Agent is responsible for continuously reading historical sales data from PostgreSQL and executing the Prophet time-series model to predict future demand per SKU. It operates on a scheduled basis (daily) and writes its predictions back to a dedicated forecasts table. Its tool set includes: `db_read_tool` (fetches historical sales records), `prophet_run_tool` (executes the forecasting model), and `db_write_tool` (persists prediction results).

Agent 2 — Decision Agent

The Decision Agent consumes the Forecasting Agent's outputs and applies reorder logic: if predicted stock falls below the defined reorder threshold within the lead-time window, it flags the SKU for procurement. It follows a structured plan → execute → verify reasoning loop:

Plan: retrieve current stock levels and forecasted demand.

Execute: compare against reorder point formula (average daily demand × lead time) + safety stock.

Verify: confirm the flag is not a duplicate of an already-open PO before escalating.

Its tool set includes: `db_read_tool` (stock levels), `forecast_read_tool` (predictions), and `po_status_check_tool` (open PO lookup).

Agent 3 — Purchasing Agent

Upon receiving a confirmed reorder flag from the Decision Agent, the Purchasing Agent drafts a formal Purchase Order, presents it to the warehouse manager for human-in-the-loop approval, and — upon approval — dispatches it via email to the pre-approved supplier. It then monitors for supplier confirmation and updates the system status accordingly.

Its tool set includes: `po_draft_tool` (generates structured PO document), `email_send_tool` (dispatches PO to supplier), `confirmation_listener_tool` (polls for reply), and `db_update_tool` (updates inventory status).

Error Handling & Resilience

Each agent implements the following failure protocols:

Email delivery failure: the Purchasing Agent retries up to three times with exponential backoff (30s, 2min, 10min); on final failure, it escalates to the human operator via an in-app notification.

Supplier non-response timeout: if no confirmation is received within 48 hours, the agent flags the PO as "Pending — Supplier Unresponsive" and notifies the manager.

Forecasting model failure: if Prophet raises an exception (e.g., insufficient data points), the Forecasting Agent falls back to a simple moving-average calculation and logs a data quality warning in the observability dashboard.

Human-in-the-Loop Checkpoint

No Purchase Order is dispatched without explicit manager approval through the dashboard. The agent presents a structured approval card showing: SKU, predicted stockout date, recommended

order quantity, supplier name, estimated cost, and reasoning trace. The manager can approve, reject, or modify the quantity before submission.

Section E: Multimodal Capabilities

SmartStock AI integrates two multimodal input channels that are directly aligned with day-to-day warehouse and procurement workflows.

E.1 — Vision / OCR: Supplier Invoice Processing

Warehouse staff frequently receive supplier invoices and delivery notes as scanned image files or photographed paper documents. The system allows users to upload these images directly through the dashboard. The uploaded image is passed to GPT-4o Vision, which extracts structured data fields — product name, SKU code, quantity received, unit price, and supplier name — and returns a JSON object.

Before any data is written to the database, the system presents the extracted fields to the user in an editable confirmation card on the dashboard. The card displays each extracted value alongside the corresponding region of the original invoice image, allowing the staff member to visually verify accuracy, correct any misread fields (e.g., a misidentified SKU or quantity), and explicitly press "Confirm & Add to Inventory" to proceed — or "Reject" to discard the extraction entirely.

Only upon confirmation does the system validate the approved data against the existing product catalogue in PostgreSQL and update inventory records. A timestamped audit log entry is created for every confirmed receipt, recording the original image, the extracted values, any manual corrections made, and the identity of the confirming user.

The full data flow is: Image Upload (React) → Base64 Encode → GPT-4o Vision API → Structured JSON extraction → Confirmation Card presented to user → Manual review & correction → User confirms → Validation against DB → Inventory update + Audit log entry

This confirmation gate eliminates the risk of AI misreads silently corrupting inventory data, reduces transcription errors, and creates a fully auditable digital record of every supplier delivery.

E.2 — Speech-to-Text: Voice-Activated NL Queries

Warehouse managers operating on the floor are often unable to type queries. The system integrates OpenAI Whisper (via API) to accept voice commands recorded through the browser's microphone. The transcribed text is fed directly into the existing Natural Language query pipeline, enabling hands-free interactions such as: "How many units of Product X do we have left?" or "Show me slow-moving items from last month."

The data flow is: Microphone input (Web Speech API / Whisper) → Transcription → NL Query Pipeline (LangChain) → DB Query → Dashboard response.

Both multimodal features are surfaced as clearly labelled input options on the dashboard UI — a microphone icon for voice and a document-upload icon for invoice scanning — ensuring they are discoverable and accessible to non-technical warehouse staff without any additional training.

Section F: Safety & Security

F.1 Human-in-the-Loop Safety Mechanism

One of the primary safety controls in SmartStock AI is the Human-in-the-Loop (HITL) approval process.

Although the Purchasing Agent can automatically generate purchase orders based on predicted stock shortages, the system does not allow fully autonomous procurement actions.

Workflow

1. The forecasting model predicts a future stock shortage.
2. The Decision Agent calculates the required reorder quantity.
3. The Purchasing Agent generates a draft Purchase Order (PO).
4. The PO is presented to an authorized manager.
5. The manager reviews and approves or rejects the request.
6. Only approved purchase orders are sent to suppliers.

Benefits

- Prevents incorrect AI-generated purchasing decisions.
- Reduces financial risks caused by forecasting errors.
- Maintains human accountability in critical business operations.
- Supports responsible AI deployment practices.

F.2 Secure API Key Management

The system relies on external AI providers such as OpenAI and Anthropic APIs. API credentials represent highly sensitive assets and must never be hardcoded within the source code.

Security Risks

Hardcoded API keys can lead to:

- Credential leakage through Git repositories.
- Unauthorized API consumption.
- Unexpected operational costs.
- Complete compromise of AI services.

Proposed Solution

All secrets are stored using environment variables and loaded at runtime.

Example:

- OPENAI_API_KEY
- ANTHROPIC_API_KEY
- DATABASE_URL
- EMAIL_SERVICE_KEY

For production environments, cloud-based secret management services can be used, such as:

- AWS Secrets Manager
- Google Secret Manager
- Azure Key Vault

Benefits

- Centralized credential management.

- Reduced exposure risk.
- Easier key rotation and maintenance.

F.3 Authentication

Authentication verifies the identity of users before granting access to system resources.

Selected Approach: JWT Authentication

SmartStock AI uses JSON Web Tokens (JWT) through Django REST Framework.

Authentication Flow

1. User submits login credentials.
2. Server validates credentials.
3. Access Token and Refresh Token are generated.
4. Access Token is attached to subsequent API requests.
5. Server validates the token before processing requests.

Advantages

- Stateless authentication.
- Scalability for distributed systems.
- Improved API security.
- Compatibility with web and mobile clients.

F.4 Authorization and Role-Based Access Control (RBAC)

Authentication answers the question "Who are you?", while authorization answers "What are you allowed to do?"

To ensure proper access control, SmartStock AI implements Role-Based Access Control (RBAC).

User Roles

Viewer

Permissions:

- View inventory information.
- Access dashboards and reports.

Restrictions:

- Cannot modify data.
- Cannot create purchase orders.

Manager

Permissions:

- Manage inventory.
- Approve purchase requests.
- Access forecasting reports.

Restrictions:

- Cannot manage system users.

Administrator

Permissions:

- Full system access.
- User management.
- System configuration.
- Security policy administration.

Benefits

- Principle of Least Privilege.
- Reduced insider threats.
- Improved auditability and accountability.

F.5 Prompt Injection Defense

Prompt Injection is one of the most significant security threats in AI-powered systems.

Threat Description

An attacker may attempt to manipulate the LLM using malicious instructions such as:

- Ignore previous instructions.
- Reveal confidential information.
- Execute unauthorized actions.

Mitigation Strategy

System Prompt Isolation

System instructions remain separate from user inputs and cannot be modified by end users.

Input Sanitization

Potentially malicious prompts are filtered and validated before reaching the LLM.

Restricted Tool Access

The LLM cannot directly access databases, email services, or administrative functions without authorization checks.

Output Validation

AI-generated outputs are verified before execution.

Benefits

- Reduced risk of unauthorized actions.
- Protection against data leakage.
- Increased reliability of AI responses.

F.6 Input Validation and Data Sanitization

All incoming data must be validated before being processed.

Risks

Poor validation may lead to:

- Invalid inventory records.
- SQL injection attacks.
- Application crashes.
- Data corruption.

Validation Measures

- Numeric range validation.
- Required field verification.
- Data type enforcement.
- Email format validation.
- Business rule validation.

Benefits

- Improved data quality.
- Increased system reliability.
- Enhanced security posture.

F.7 Rate Limiting and Abuse Prevention

AI endpoints are particularly vulnerable to excessive usage and automated abuse.

Risks

- Denial of Service (DoS) attacks.
- Excessive API costs.
- Performance degradation.

Proposed Controls

- User-based request throttling.
- IP-based request limits.
- Daily AI usage quotas.
- Request monitoring and anomaly detection.

Example Limits

- 100 requests per minute per user.
- Daily token consumption limits.
- Maximum concurrent requests.

F.8 Personally Identifiable Information (PII) Protection

The system stores business-related information that may contain personally identifiable data.

Examples

- Supplier names.
- Email addresses.
- Phone numbers.
- Business financial information.

Protection Mechanisms

- Database encryption.
- Role-based access restrictions.

- Audit logging.
- Data retention policies.
- Secure backups.

Benefits

- Regulatory compliance.
- Reduced privacy risks.
- Increased stakeholder trust.

F.9 Security Monitoring and Audit Logging

All critical system actions are logged for monitoring and investigation purposes.

Logged Events

- Login attempts.
- Permission changes.
- Purchase order approvals.
- AI-generated actions.
- System configuration changes.

Benefits

- Incident investigation.
- Security auditing.
- Compliance reporting.
- Threat detection.

F.10 Conclusion

The SmartStock AI security architecture combines traditional cybersecurity practices with AI-specific safety controls. Through JWT authentication, RBAC authorization, API key protection, prompt injection defense, input validation, rate limiting, PII protection, and Human-in-the-Loop approval workflows, the platform provides a secure and reliable environment for inventory management and AI-assisted decision making.

Section G: Observability & Evaluation

SmartStock AI integrates a comprehensive observability and evaluation framework to ensure the reliability, transparency, and continuous improvement of all AI components across the LLM, RAG, and Agent layers.

G.1 — Observability Tooling: Langfuse

The system integrates Langfuse as its primary AI observability platform. Langfuse is connected to the LangChain pipeline via a callback handler, meaning every LLM call, RAG retrieval step, and agent tool invocation is automatically traced without requiring manual instrumentation in each function.

Each trace captures: the input prompt and system message, the retrieved RAG context chunks and their source references, the LLM's raw output, token consumption (prompt tokens + completion tokens), response latency in milliseconds, and the total API cost per call. All traces are stored and accessible through the Langfuse dashboard, allowing the team to inspect individual requests, identify failure patterns, and monitor cost trends over time.

For the Purchasing Agent specifically, Langfuse records the full multi-step trace of each agent run — from the initial stockout detection trigger, through each tool call (database read, PO draft, email dispatch), to the final system status update — providing a complete audit trail for every automated procurement action.

G.2 — Evaluation Metrics

Three core metrics are defined and measured continuously:

Metric 1 — Retrieval Precision@5 (RAG pipeline): For every user query processed through the RAG pipeline, the system evaluates whether the top 5 retrieved document chunks are genuinely relevant to the query. A golden evaluation set of 30 annotated queries is used, where each query has a pre-labelled set of expected relevant chunks. Precision@5 is calculated as the ratio of relevant chunks among the top 5 retrieved. Target baseline: ≥ 0.80 .

Metric 2 — Answer Faithfulness (LLM output): This metric measures whether the LLM's generated response is grounded in the retrieved context rather than hallucinated from world knowledge. It is evaluated using LangChain's built-in faithfulness evaluator, which cross-checks each claim in the response against the provided context chunks. Any response scoring below 0.75 faithfulness is flagged for human review. Target baseline: ≥ 0.85 .

Metric 3 — Agent Task Success Rate (Purchasing Agent): This metric tracks the ratio of Purchase Order drafts that are approved by the warehouse manager without modification versus those that are rejected or manually corrected. A high approval rate indicates the agent is generating accurate, well-structured POs. A declining rate is an early signal of data quality issues (e.g., incorrect supplier records or outdated pricing). Target baseline: ≥ 0.90 approval without modification.

G.3 — Golden Evaluation Dataset

A curated golden dataset of 30 annotated test cases is constructed to enable automated regression testing of the NL-to-query pipeline. Each test case contains: a natural language input query (e.g., "Show me products with less than 10 units in stock"), the expected system action or

database filter, and the expected response structure. The dataset covers five query categories: stock level checks, slow-moving item identification, supplier lookup, reorder status queries, and demand forecast summaries. This dataset is version-controlled in the project repository and executed as part of the CI/CD pipeline on every merge to main, ensuring that model or prompt changes do not silently degrade query accuracy.

G.4 — Runtime Monitoring & Alerting

The following alerting thresholds are configured in Langfuse to notify the team of degrading system health:

LLM response latency p95 exceeding 3 seconds triggers a warning — indicating either model overload or excessive context window size.

LLM API error rate exceeding 1% over a 1-hour rolling window triggers an alert — indicating authentication failures or rate limit exhaustion.

Daily token spend exceeding a pre-defined budget cap triggers a cost alert — preventing unexpected API billing overruns during peak usage.

Agent task success rate dropping below 0.80 over a 24-hour window triggers an operational review flag.

All alerts are delivered via email notification to the team lead and surfaced as a warning banner in the SmartStock dashboard for the warehouse manager.

Section H: Engineering & Delivery

H.1 Proposed Technology Stack

The following technology stack has been selected for SmartStock AI:

Type	Finding / Comment
 Frontend	React.js — Recharts for data visualisation, Tailwind CSS for styling.
 Backend	Django REST Framework — RESTful API layer with role-based endpoint access.
 Database	PostgreSQL (structured operational data) + Redis (caching and session management).
 AI / ML	Scikit-learn & Prophet (time-series demand forecasting) + LangChain (LLM and RAG orchestration).
 Primary Risk	Data Quality & Sparsity: clean historical sales data required for model accuracy; real-time warehouse/dashboard synchronisation must be maintained.

Overview

The engineering architecture of SmartStock AI is designed to provide scalability, maintainability, reliability, and ease of deployment. The system follows a modern full-stack architecture that combines web technologies, machine learning models, cloud deployment practices, and AI orchestration frameworks.

The selected technology stack enables efficient inventory management, demand forecasting, AI-assisted analytics, and autonomous purchasing workflows while maintaining flexibility for future expansion.

H.2 Technology Stack

Frontend Layer

React.js

React is used to build a responsive and component-based user interface.

Responsibilities:

- Dashboard rendering.
- Inventory management screens.
- Analytics visualization.
- Purchase order workflows.
- AI assistant interaction.

Recharts

Used for visual analytics and forecasting dashboards.

Examples:

- Inventory trends.
- Demand forecasts.
- Supplier performance charts.
- Sales analytics.

Tailwind CSS

Used for styling and responsive design.

Benefits:

- Faster UI development.
- Consistent design system.
- Mobile responsiveness.

H.3 Backend Layer

Django REST Framework (DRF)

The backend exposes RESTful APIs and contains the business logic of the system.

Responsibilities:

- User management.
- Authentication and authorization.
- Inventory operations.
- Supplier management.
- Purchase order workflows.
- AI service integration.

Benefits

- Rapid development.
- Strong security features.
- Mature ecosystem.
- Excellent API support.

H.4 Database Layer

PostgreSQL

PostgreSQL serves as the primary relational database.

Core Tables:

- Users
- Products
- Categories
- Suppliers
- Inventory Records
- Purchase Orders
- Forecast Results
- Audit Logs

Why PostgreSQL?

- High reliability.
- ACID compliance.
- Strong analytical capabilities.
- Compatibility with future RAG extensions through pgvector.

Redis

Redis is used as an in-memory data store.

Responsibilities:

Caching

Stores frequently accessed information to reduce database load.

Session Storage

Supports user session management.

Task Queues

Facilitates background processing and asynchronous operations.

Benefits

- Faster response times.
- Reduced database pressure.
- Improved scalability.

H.5 Artificial Intelligence Layer

Prophet Forecasting Model

The forecasting subsystem uses Prophet to predict future inventory demand.

Inputs:

- Historical sales data.
- Seasonal trends.
- Business cycles.

Outputs:

- Future demand estimates.
- Predicted stock shortages.
- Recommended reorder points.

Challenges

Forecast accuracy depends heavily on data quality and quantity.

Mitigation

- Data cleaning pipelines.
- Outlier detection.
- Periodic model retraining.

LangChain Framework

LangChain acts as the orchestration layer between the LLM, business tools, and enterprise data.

Responsibilities:

- Prompt management.
- Tool calling.
- Agent orchestration.
- RAG integration.
- Memory management.

Benefits

- Modular architecture.
- Easier AI integration.

- Support for future agent expansion.

H.6 Containerization Strategy

Problem

Development environments often differ between team members. This creates the classic problem: "Works on my machine."

Solution: Docker

Each service runs inside an isolated container. Containers include:

- Frontend Container
- Backend Container
- PostgreSQL Container
- Redis Container

Benefits

- Environment consistency.
- Simplified deployment.
- Faster onboarding.
- Improved portability.

Docker Compose

Docker Compose orchestrates all services together.

Responsibilities:

- Service networking.
- Environment configuration.
- Startup sequencing.
- Local development automation.

H.7 Deployment Strategy

Frontend Deployment

The React application can be deployed on:

- Vercel
- Netlify

Reasons:

- Automated deployments.
- Global CDN support.
- Easy GitHub integration.

Backend Deployment

The Django backend can be deployed on:

- Render
- Railway

Reasons:

- Simple setup.
- Managed infrastructure.
- Automatic scaling capabilities.
- Cost-effective student-friendly plans.

H.8 Continuous Integration and Continuous Deployment (CI/CD)

Problem

Manual testing and deployment are slow and error-prone.

Solution

GitHub Actions automates the development pipeline.

CI Pipeline

Executed on every push:

1. Code checkout.
2. Dependency installation.
3. Linting.
4. Unit testing.
5. Integration testing.
6. Build verification.

CD Pipeline

After successful validation:

1. Build application.
2. Deploy backend.
3. Deploy frontend.
4. Run post-deployment checks.

Benefits

- Faster releases.
- Reduced deployment errors.
- Improved software quality.

H.9 Testing Strategy

Testing ensures system reliability and maintainability.

Unit Testing

Tests individual components independently.

Examples:

- Forecasting functions.
- Business rules.
- Utility services.

Integration Testing

Tests communication between components.

Examples:

- Frontend ↔ Backend.
- Backend ↔ Database.
- Backend ↔ AI Services.

End-to-End Testing

Validates complete business workflows.

Example Scenario:

1. User updates inventory.
2. Forecasting model detects shortage.
3. Purchase order is generated.
4. Manager approves request.
5. Supplier receives order.

H.10 Documentation Strategy

Proper documentation improves maintainability and onboarding.

README Documentation

Includes:

- Installation steps.
- Environment setup.
- Deployment instructions.
- Project structure.

API Documentation

Generated using OpenAPI and Swagger.

Benefits:

- Easier API consumption.
- Faster frontend integration.
- Improved developer experience.

Architecture Documentation

Includes:

- System diagrams.
- Database schema.
- AI workflow diagrams.
- Deployment architecture.

H.11 Engineering Risks and Mitigation

Risk 1: Poor Data Quality

Impact:

- Inaccurate forecasting.

Mitigation:

- Data validation.
- Cleaning pipelines.
- Monitoring dashboards.

Risk 2: Scalability Challenges

Impact:

- Slow response times.

Mitigation:

- Redis caching.
- Query optimization.
- Horizontal scaling.

Risk 3: AI Operational Costs

Impact:

- Increased expenses.

Mitigation:

- Token monitoring.
- Request caching.
- Efficient prompt engineering.

Risk 4: Service Failures

Impact:

- Reduced availability.

Mitigation:

- Automated backups.
- Health monitoring.
- Disaster recovery plans.

H.12 Conclusion

The SmartStock AI engineering architecture combines modern web development, machine learning, AI orchestration, and cloud-native deployment practices. Through React, Django REST Framework, PostgreSQL, Redis, Prophet, LangChain, Docker, and GitHub Actions, the platform achieves scalability, maintainability, reliability, and production readiness while supporting future AI-driven enhancements.